

Laborator 9

Implementarea sistemelor în timp real Nucleul de timp real (Real time kernel)

Obiectiv Laboratorul se dorește o aplicare practică la cele prezentate la curs legate de organizarea unui nucleu al unui sistem de operare în timp real: lista de task-uri, crearea și ștergerea task-urilor din lista de task-uri. Tot aici se va prezenta și modul în care este definit timpul de execuție (de calcul) pentru fiecare task în parte ca fiind alcătuit din unități de timp elementare. În acest laborator se va prezenta un exemplu de sistem de operare care folosește o listă statică precum și a funcțiilor de adăugarea task-urilor în listă iar fiecare task va avea propriul timp alocat în care acesta va fi rulat.

Breviar teoretic

Un nucleu în timp real este constituit din lista de task-uri și din funcțiile necesare accesării acestei liste.

Lista de task-uri reprezintă o colecție de obiecte task, fiecare obiect (tip de date definit) conține un pointer către stiva hard, un pointer către stiva soft, stiva hard și stiva soft, timpul în unități elementare în care task-ul rulează și gradul de prioritate.

Așa cum a fost arătat la curs, listele de task-uri pot fi declarate sub forma unor vectori de lungime fixă pe tot parcursul programului, lungime fixă determinată de la începutul execuției programului, sau sub forma unor liste dinamice unidirecționale, folosind în acest caz alocarea dinamică a memoriei.

Prima metodă are avantajul simplității în descriere, dar necesită lucrul cu un vector supraevaluat ca dimensiune la numărul maxim de task-uri aflate în starea Ready la un moment dat.

Soluția cu alocarea statică pentru lista de task-uri este aplicată în cazul în care se lucrează cu un număr fix de task-uri pe tot parcursul programului.

Alocarea dinamică a listei se aplică în situațiile în care numărul de task-uri care sunt folosite la un moment dat variază. În această situație se impune o exploatare dinamică a listei în sensul în care să se permită adăugarea sau ștergerea din listă a unui element oriunde în program. Dimensiunea listei este așadar în acest caz variabilă pe parcursul programului care rulează permițând exploatarea resurselor eliberate în timpul programului.

În oricare din situațiile de mai sus, lista cu task-uri este inițial vidă deci este nevoie de o funcție pentru adăugarea unui task în listă. De asemenea se dorește ștergerea acestuia atunci când nu mai este utilizat. Deci trebuie declarate două funcții care se ocupă cu managementul task-urilor.

Durata fiecărui task se măsoară în unități elementare de timp, care sunt divizibile cu timpii de calcul. O unitate elementară se obține de la circuitul periferic timer, bineînțeles după ce acesta a fost inițializat corespunzător și a primit o constată de timp.

Desfășurarea lucrării

Se cere proiectarea unui program pentru un sistem în timp real, care să realizeze pentru început următoarele sarcini:

- contorizarea timpului: ore : minute :secunde;
- afișarea timpului la fiecare schimbare a secunde.

Se va introduce următorul program:

```
#include <8051f120.h> //fișier pentru declaratii
#include <stdio.h>
#define MAX_TASK 2
//declaratii LED
sbit LED = P1^6;
sbit TASTA = P3^7;
#define SYSCLK 3062500

//variabile globale unice
unsigned char contor_delay=0;//pentru temporizarea de la ceas
//variabile globale folosite in program de catre nucleul de operare
unsigned char task_crt=0;//task curent
unsigned char task_old=1;//task anterior
unsigned char timp_crt=0;//timpul task-ului curent

unsigned char task_varf=0;
//variabile folosite de program
unsigned char sem_afisare=0;
unsigned int ore=1,minute=30,secunde=0;
unsigned int point;//pentru conversie pointer functie

//declaratia unui task
typedef struct Task
{
    unsigned char stiva_hard[16];//stiva hardware
    //unsigned char stiva_soft[8];//stiva software - set de variabile folosit
    unsigned char pstiva_hard;
    //unsigned char pstiva_soft;
    unsigned char timp;
}task;

//declaratie lista de task-uri
task taskuri[MAX_TASK];

//task 1
void task1(void)
{
    for(;;)
```

```

{
    if(!contor_delay)
    {
        secunde++;
        if(secunde==60)
        {
            secunde=0;
            minute++;
            if(minute==60)
            {
                minute=0;
                ore++;
            }
        }
    }
    contor_delay=100;
    sem_afisare=1;
}
}
}

```

```

//task 2
void task2(void)
{
    for(;;)
    {
        if(sem_afisare)
        {
            sem_afisare=0;
            printf("\r%2d:%2d:%2d",ore,minute,secunde);
        }
    }
}

```

```

//functie care adauga un task la o lista de task-uri
void adauga(unsigned char timp,void* functie)
{
    if(task_varf<MAX_TASK)
    {
        point = (unsigned int) functie;//conversie adresa functie
        taskuri[task_varf].stiva_hard[1]=(unsigned char) ((point>>8)&0x00ff);//salvare adresa
        stiva
        taskuri[task_varf].stiva_hard[0]=(unsigned char) (point&0x00ff);
        taskuri[task_varf].pstiva_hard= (taskuri[task_varf].stiva_hard)+5;//salvare stiva
        taskuri[task_varf].timp=timp;
        task_varf++;
    }
}

```

```
}
```

```
//functii initializare
```

```
//initializare sistem
```

```
void Init_System(void)
```

```
{
```

```
//dezactivare timer watch-dog
```

```
WDTCN = 0xde;//succesiune de secvente pentru dezactivare watchdog
```

```
WDTCN = 0xad;
```

```
}
```

```
//initializare canale timer 0 si 1
```

```
void Init_TMR(void)
```

```
{
```

```
SFRPAGE = TIMER01_PAGE;//pregateste pagina pentru configurare timer 0 si 1
```

```
TCON = 0x50;//activare canal timer 0 si timer 1
```

```
TMOD = 0x21;//temporizator canal 1 mod 2 (8 biti cu revenire), canal 0 mod 1 (16 biti fara revenire)
```

```
CKCON = 0x10;//selectie sursa ceas timer 1 - fara prescaler 12
```

```
TH1 = 0xfb;// - comunicare 19200bps
```

```
TH0 = 0x9c;//divizare pentru a obtine 100 herti (10 ms) la timer 0
```

```
TL0 = 0x50;//
```

```
}
```

```
//initializare porturi I/O
```

```
void Init_PORT(void)
```

```
{
```

```
SFRPAGE = CONFIG_PAGE;//pagina pentru configurare porturi
```

```
P0MDOUT = 0x03;//configureaza pinul TX ca iesire
```

```
XBR0 = 0x04;//configureaza portul UART0 pe porturile 0
```

```
//configurare port 2
```

```
XBR2 = 0x40;//configureaza liniile de port
```

```
P1MDOUT |= 0x40;//configureaza pinul P1.6 ca iesire (LED)
```

```
}
```

```
//initializare canal serial
```

```
void Init_UART0(void)
```

```
{
```

```
SFRPAGE = UART0_PAGE;//setare pagina SFR pentru canalul 0 UART
```

```
SCON0 = 0x50;//serial modul 1 receptie activata
```

```
TI0 = 1;//initializare transmisie seriala
```

```
}
```

```
//activare intreruperi
```

```
void enable_interrupt(void)
```

```
{
```

```

SFRPAGE = LEGACY_PAGE;
//activare intrerupere
IE = 0x82;
//initializare pagina seriala
SFRPAGE = UART0_PAGE;
}

```

```

void main(void)
{
    Init_System();//initializare sistem
    Init_TMR();//initializare canale timer
    Init_PORT();//initializare porturi I/O
    Init_UART0();//initializare canale seriale

    //adauga cele doua task-uri la lista de task-uri
    adauga(10,task1);
    adauga(10,task2);

```

```

//initializare stiva pentru task 1
SP = (taskuri[0].stiva_hard)-1;
timp_crt=taskuri[0].timp;

```

```

enable_interrupt();//activare intreruperi

```

```

//pornire task 1 initial
task1();
for(;;);

```

```

}

```

```

//subrutina tratare timer
void Timer0_ISR (void) interrupt 1
{
    TH0 = 0xf6;//divizare cu 250 - comunicare 9600bps
    TL0 = 0x08;//divizare pentru a obtine 100 herti (10 ms)

```

```

//contor pentru timp
if(contor_delay)
    contor_delay--;

```

```

//contor pentru terminarea timpului de executie al unui task curent
if(timp_crt)

```

```

        timp_crt--;
else
{
//comutare task-uri
task_old=task_crt;
task_crt++;
if(task_crt==2)
    task_crt=0;

//comutare stive soft si hard
taskuri[task_old].pstiva_hard=SP;

SP = taskuri[task_crt].pstiva_hard;

}

}

//subrutina tratare intrerupere seriala
void Serial0_ISR (void) interrupt 4
{
//receptie caracter serial?
if(RI0)
{
//tratare receptie seriala

RI0=0;
}
}

```

Aplicații. Probleme

1. Se va introduce programul și se va rula. Se va realiza conexiunea cu calculatorul prin Hiperterminal, conexiunea se realizează la 19200, 8 biți date, fără paritate, 1 bit stop.
2. Care este numărul maxim de task-uri care pot fi adăugate la sistem în faza inițială?
3. Să se analizeze funcția care realizează adăugarea unui task. Ce reprezintă parametrul 2 al funcției? De ce este preluat de o variabilă locală de timp unsigned int?
4. Modificați timpul inițial astfel încât acesta să indice chiar ora exactă.
5. Care este valoarea timpului de execuție pentru task-ul 1 și task-ul 2? Să se modifice timpul de execuție pentru task-ul 2 la 1 secundă.

6. Să se mai adauge un task care să realizeze tratarea tastei și contorizarea apăsării acesteia. În task-ul 2 (care realizează afișarea) se va afișa la fiecare secundă pe lângă timp și valoarea contorului tastei.